

Real-time tracking and counting of grape clusters in the field based on channel pruning with YOLOv5s

Lei Shen^{a,b,c}, Jinya Su^d, Runtian He^{a,b,c}, Lijie Song^{a,b,c}, Rong Huang^e, Yulin Fang^e,
Yuyang Song^e, Baofeng Su^{a,b,c,*}

^a College of Mechanical and Electronic Engineering, Northwest A&F University, Yangling, Shaanxi 712100, China

^b Key Laboratory of Agricultural Internet of Things, Ministry of Agriculture and Rural Affairs, Yangling, Shaanxi 712100, China

^c Shaanxi Key Laboratory of Agricultural Information Perception and Intelligent Services, Yangling, Shaanxi 712100, China

^d Department of Computing Science, University of Aberdeen, Aberdeen AB24 3UE, UK

^e College of Enology, Northwest A&F University, Yangling, Shaanxi 712100, China

ARTICLE INFO

Keywords:

Grape
YOLOv5s
Channel pruning
Multiple object tracking
Real-time counting

ABSTRACT

Accurate fruit counting helps grape wine industry make better logistics and decisions before harvest, and therefore produce higher quality wine. In view of poor real-time performance of the existing fruit tracking and counting methods, and a lack of effective counting methods for cluster-like fruits due to their huge shape variabilities. In this study, an end-to-end lightweight counting pipeline is developed to automate the processing of video data for real-time tracking and counting of grape clusters in field conditions. First, based on channel pruning algorithm, a more lightweight YOLOv5s cluster detection model is obtained, where number of model parameters, model size and floating-point operations (FLOPs) are reduced by 79 %, 76 %, and 58 %, respectively, and the pruned model size is only 3.4 MB. Secondly, the soft non-maximum suppression is introduced in prediction stage to improve detection performance for clusters with overlapping grapes. Test results show that mAP reaches 82.3 % and average inference time is 6.1 ms per image, which effectively reduces model parameters and complexity while ensuring detection accuracy. Finally, online multiple object tracking of clusters is implemented by integrating the detection results and SORT algorithm, where two counting modes are set by introducing counting lines. Test results on 8 videos indicated that the average counting accuracy of the proposed method reached 84.9 %, correlation coefficient with manual counting reached 0.9905, and speed of video processing reached up to 50.4 frames per second (FPS), meeting field real-time requirements. This study provides a timely technical reference for the development of orchard robots to achieve real-time automated yield estimation and accurate crop management decisions.

1. Introduction

A major challenge of wine industry historically has been the accurate estimation of vineyard yields and winemaking potential, i.e., yield forecasting before harvest. Currently, this remains an unresolved challenge with significant technical and economic implications (Barriguiha et al., 2021; Dunn and Martin, 2004; Laurent et al., 2021). As grape acreage and yields expand, accurate estimation of wine grape yields will help winemaking industry make better logistics and decisions and produce higher quality wines (Fernández-González et al., 2011). Additionally, growers are also able to more precisely plan storage and sale of fruit, which is critical to improving winery efficiency and wine

marketing strategies (Dorj et al., 2017).

Current methods to estimate vineyard productivity at an industrial scale are destructive and labor-intensive, and lack of economic feasibility (Herrero-Huerta et al., 2015). In addition, some methods are mainly based on manual visual inspection of vineyards, where experts only sample small areas of large vineyards and extrapolate the results to the whole area, with no guarantee of reliability and accuracy (Zabawa et al., 2020). These methods are cumbersome and insufficient to obtain representative yield data (Aquino et al., 2018). Therefore, there is an urgent need to develop an accurate and efficient method for counting grape clusters in field conditions to automate orchard yields estimation.

Accurate and rapid fruit detection in the field is the basis for

* Corresponding author.

E-mail address: bfs@nwsuaf.edu.cn (B. Su).

<https://doi.org/10.1016/j.compag.2023.107662>

Received 10 August 2022; Received in revised form 10 October 2022; Accepted 14 January 2023

Available online 1 February 2023

0168-1699/© 2023 Elsevier B.V. All rights reserved.

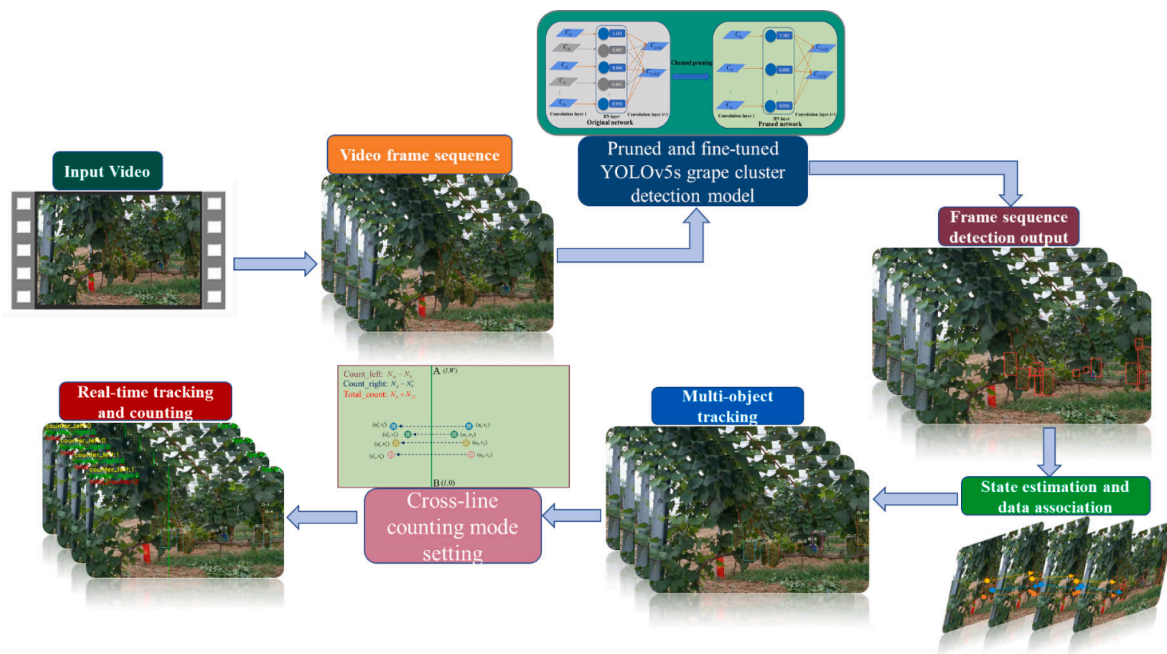


Fig. 1. The proposed tracking and counting pipeline based on video data.

automated counting. With the advent of inexpensive sensors and electronic systems, off-the-shelf RGB cameras and computer vision systems based on advanced deep convolutional neural networks (CNNs) can provide affordable and versatile solutions for fruit detection (Kamilaris and Prenafeta-Boldú, 2018; LeCun et al., 2015). For example, Bargoti and Underwood (2017) used a representative two-stage regional CNN, Faster R-CNN (Ren et al., 2015; Yi et al., 2021), to detect mangoes, almonds, and apples. Similarly, it has also been used to detect coconuts in complex backgrounds to determine their maturity stages (Parvathi and Selvi, 2021). Further, Mask R-CNN (He et al., 2017) instance segmentation network developed on the basis of Faster R-CNN has also been successfully used to detect apples and grapes in the field (Santos et al., 2020; Shen et al., 2022; Wang and He, 2022). Compared with two-stage detection networks, one-stage object detection network YOLO (You Only Look Once) has the advantages of small size and fast detection, and is widely used for detection of tomatoes, kiwis, table grapes, and mangoes in complex field environments (Fu et al., 2021; Koirala et al., 2019; Liu et al., 2020). These studies have well demonstrated the potentials of CNNs in fruit recognition. However, most of these deep learning-based methods are mainly used to detect fruits in single images, which are only applicable to high-performance detection platforms due to their model and parametric sizes (Wu et al., 2020). This severely affects the deployment and application of fruit detection models on small mobile terminals and limits the development of orchard robots.

Several studies have shown that deep CNN-based detection results can be integrated by data association to count fruits in orchards through object tracking or mapping (Liu et al., 2019). To address the problem of video-based fruit counting, Vasconez et al. (2020) achieved the counting of three fruits using Faster R-CNN with InceptionV2 combined with Gaussian estimation-based multiple object tracking, where the counting performance ranged from 44 % to 93 %. Gao et al. (2022), based on YOLOv4-tiny and trunk tracking, successfully detected and counted apples in videos. However, in the above studies, fruit tracking is lost due to fruit occlusion or sudden movement of camera platform, which is prone to double counting. And this problem can be effectively solved by multi-view method or 3D reconstruction. In this regard, Stein et al (2016) developed a multi-view approach based on Faster R-CNN to detect fruits on mango trees and applied trajectory data provided by the navigation system to establish the correspondence between images for

fruit counting, and the obtained counting accuracy was compared with ground truth with an error of 1.36 %. Exceptionally, Liu et al. (2018) proposed a novel fruit counting pipeline using a fully convolutional network (FCN) to segment video frame image into fruit and non-fruit pixels. The fruit is then tracked across frames using a Hungarian algorithm that combines the tracking results with a structure of motion (SfM) algorithm to compute relative 3D position and size estimates to reject outliers and double-counted fruit trajectories. Although these methods can achieve high counting accuracy, they are usually time-consuming and therefore are difficult to meet the demand for real-time counting. Compared to fruits such as apples and mangoes with more regular shapes, for cluster-like fruits, grape inter-cluster shapes exhibit great variabilities, and it is still a challenge to achieve tracking counts for clusters via video.

To address the above-mentioned challenges, this study develops a lightweight counting pipeline based on channel pruning of YOLOv5 and SORT algorithm, which enables real-time tracking and counting of grape clusters in the field based on video data. The proposed pipeline visualization is given in Fig. 1. The main contributions are summarized as follows.

- (1) YOLOv5s model was pruned based on channel pruning algorithm, which effectively reduces model parameters and complexity while ensuring detection accuracy.
- (2) Soft non-maximum suppression (Soft-NMS) was introduced in the prediction stage to enhance model detection performance for overlapping grape clusters.
- (3) SORT algorithm was drawn to achieve online multiple object tracking of grape clusters.
- (4) Counting line was proposed in the video, and two counting modes were set, which effectively alleviated counting error caused by ID switching during tracking process. Tested on 8 videos, the average accuracy rate reached 84.9 %, and the processing speed reached up to 50.4 frames per second (FPS).

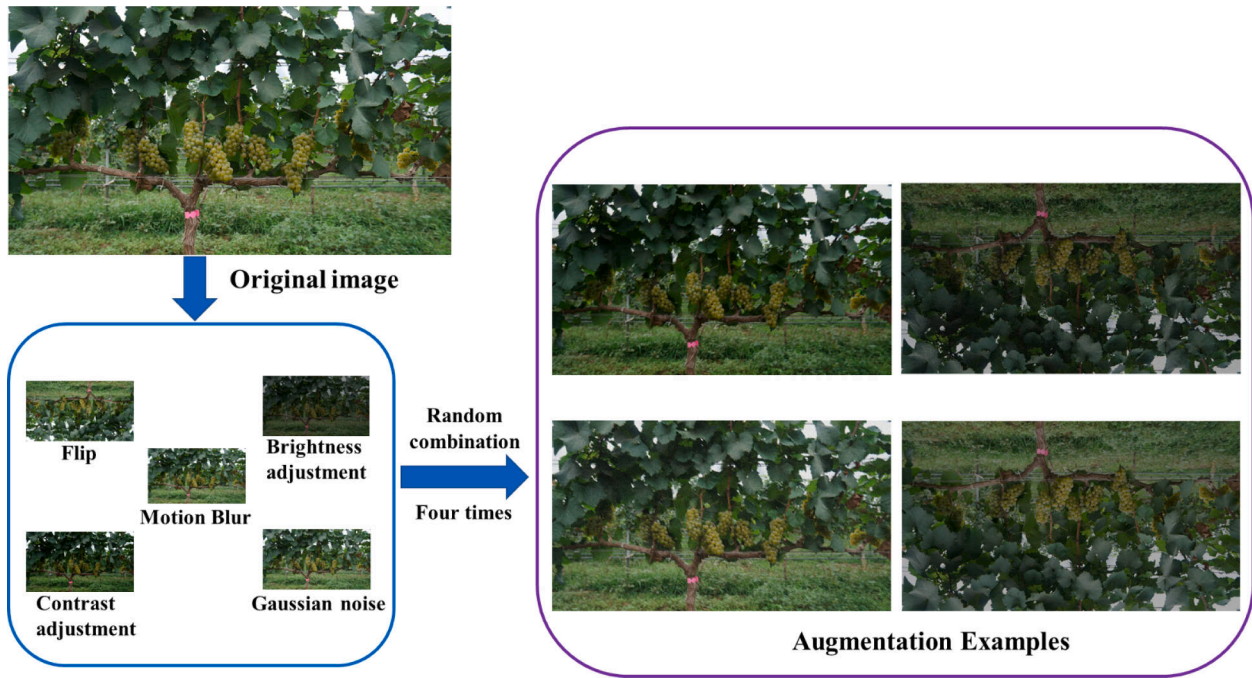


Fig. 2. Dataset augmentation process.

2. Materials and methods

2.1. Data preparation

2.1.1. Data acquisition and annotation

Grape cluster (Chardonnay wine grape variety) images and video datasets used for the experiment were collected at a wine grape cultivation site in Yangling District, Shaanxi Province, China. The grapevines were planted in rows from north to south with a spacing of about 3 m between rows. The image dataset was collected by using a digital camera (SONY ILCE-5100L) with an image size of 3008×1668 pixels. 218 images were collected in 2020 (with acquisition distance between 0.5 and 1 m) and 464 images in 2021 (with acquisition distance between 1 and 1.5 m), both from July to September before grape harvest, in different weather and lighting conditions. The video dataset was collected before harvest in 2021 by using a remote-controlled field

vehicle mounted with a digital camera. The vehicle dataset was collected randomly by row at a time, containing different directions of travel (south to north or north to south), and the recording was done with the vehicle at a horizontal distance of about 1.5 m from the grapevines, the camera at a vertical height of 1.2 m above the ground, and the movement speed of about 0.5–1 m/s. A total of 8 videos of different lengths were collected, named from V1 to V8 respectively, where video capture was at a frame rate of 30 FPS.

Considering the huge variations of grape cluster shapes in the field and the problems of occlusion, it is difficult to annotate accurately using rectangular boxes directly. Therefore, the acquired images were annotated with Labelme (Russell et al., 2008) polygon annotation tool, which can achieve accurate annotation of cluster outlines, with a total of 6227 grape cluster instances. After annotation, the external rectangular boxes were automatically calculated based on grape cluster polygon outlines, which were uniformly converted to YOLO standard dataset format for

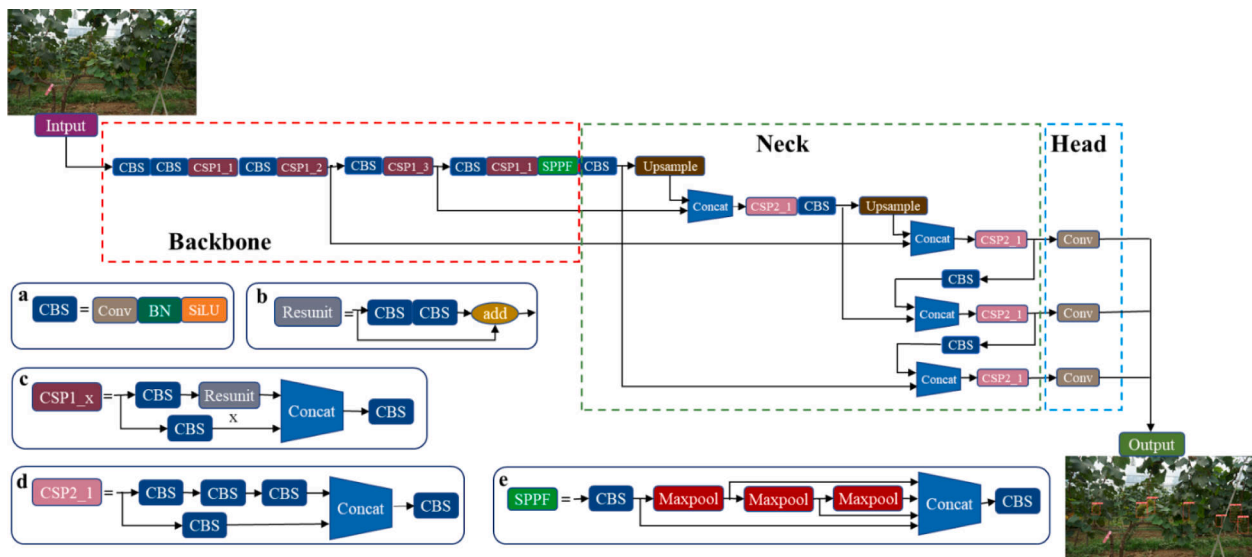


Fig. 3. Structure of grape clusters detection network based on YOLOv5s.

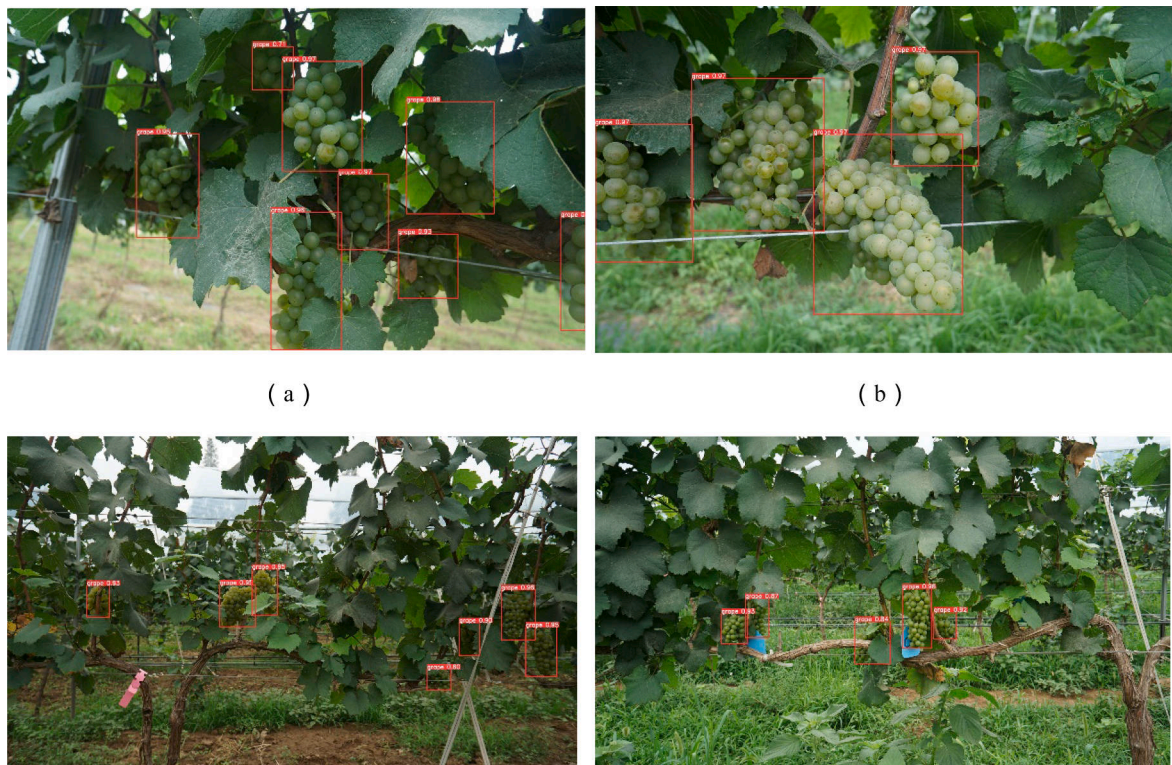


Fig. 4. (a) and (b) are example results of testing the dataset collected in 2020. (c) and (d) are example results of testing the dataset collected in 2021.

training and testing. The completed annotated datasets of different years were divided by 8:2, and finally 544 training set pairs and 138 test set pairs were obtained. The detailed image dataset building process can be found in our previous work (Shen et al., 2022).

2.1.2. Image augmentation

To enrich training set and prevent overfitting, random combination of enhancement was performed four times by using random flip (horizontal or vertical), brightness adjustment, motion blur, contrast adjustment and adding Gaussian noise. The corresponding annotation file of each image is also transformed, and the augmentation process is shown in Fig. 2. The augmented training set is composed of 2720 images.

2.2. Rapid grape clusters detection by channel pruning YOLOv5s

In this section, firstly, YOLOv5s grape cluster detection model is obtained by transfer learning training on the augmented image dataset. Second, to further simplify the model and improve its detection efficiency, YOLOv5s model is pruned based on channel pruning algorithm to reduce model parameters and complexity as well as to decrease running memory occupation. This allows faster and more accurate detection of grape clusters, and facilitates model deployment. The details will be explained in the following subsections.

2.2.1. Grape clusters detection by YOLOv5s.

YOLO is a typical one-stage object detection algorithm, where its main idea is to input model with whole image and directly regress object coordinates and classification probability of the target using CNN (Redmon et al., 2016). This end-to-end detection network greatly improves model detection speed (Tian et al., 2019). YOLOv5s, as a network with the smallest depth and the smallest width of feature map in YOLOv5 series, has a small model size, which is convenient for subsequent expansion applications (Jocher et al., 2020). Therefore, the advanced YOLOv5s network is selected as the baseline network for grape

clusters detection in this study, and the main model structure is shown in Fig. 3, which includes input, backbone, neck, head and loss function, detailed as below.

- (1) Input: mosaic data enhancement is used, and four images are stitched by random scaling, random cropping, and random scheduling to improve network robustness. In addition, for different object sizes in datasets with different acquisition distances, adaptive anchor box technique is used to automatically find the most suitable anchor box length and width through the network.
- (2) Backbone: using CSPNet (Wang et al., 2020) structure to extract information-rich features from input image.
- (3) Neck: using FPN + PAN structure for the aggregation of three scale semantic features to recognize target grape clusters of different sizes.
- (4) Head: 1×1 convolution structure is used to output three sets of feature maps with different scales. Apply anchor boxes to output feature map to generate a final output vector with category probabilities, confidence scores and bounding boxes.
- (5) Loss function: representation of the difference between prediction and ground truth during training phase. A Complete IoU (CIoU) loss (Zheng et al., 2020) is used as the loss function of the bounding box.

Detailed module description is given as below:

(a) **CBS** (Fig. 3a): A module consisting of a convolutional layer, a batch normalization layer and a SiLu activation function, which is the most basic structure in YOLOv5s network.

(b) **CSP**: Evolved from CSPNet network, two CSP network structures are designed in YOLOv5s. CSP1_x (Fig. 3c) is applied to backbone and consists of CBS module, x residual units (Fig. 3b) and a concatenate function. The other one, CSP2_1 (Fig. 3d), is applied to Neck and consists of a CBS module and a concatenation function to enhance network feature fusion.

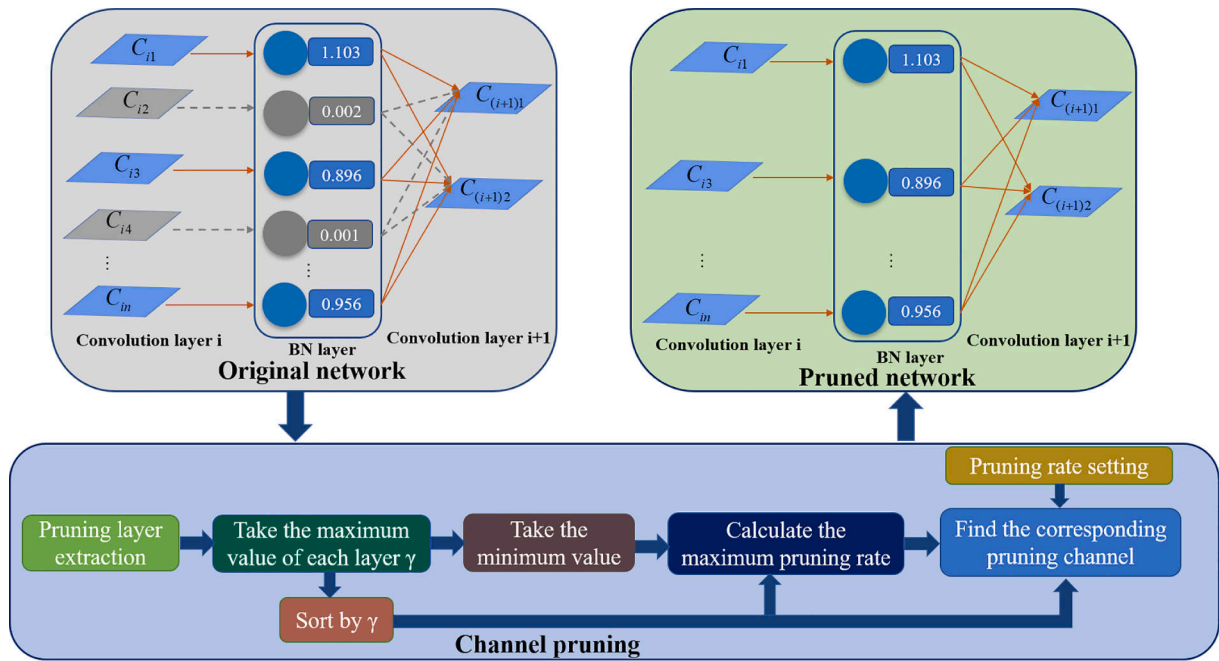


Fig. 5. Main process of channel pruning.

(c) **SPPF** (Spatial pyramid pooling-fast) (Fig. 3e): Multi-scale feature fusion is performed by specifying a convolutional kernel and using output after each pooling as input for the next pooling. As an upgraded version of SPP (Spatial pyramid pooling), the original parallel max pooling is replaced by serial to improve model efficiency.

During model training, this study fine-tuned network initialization by loading the pre-trained YOLOV5s model weights on the COCO image dataset (Lin et al., 2014) to speed up model convergence. The input size was set to 640×640 . The learning rate, batch size and epoch were set to 0.01, 16 and 200, respectively. SGD optimizer was used to iteratively optimize network parameters. The detection results after model fine-tuning are shown in Fig. 4.

2.2.2. Channel pruning for YOLOV5s grape clusters detection model

Although the fine-tuned YOLOV5s can accurately detect grape clusters under field conditions, model pruning is necessary to reduce model complexity, save computational cost and memory usage, and facilitate application to subsequent real-time tracking and counting. The fine-tuned YOLOV5s grape clusters detection model is pruned by using channel pruning algorithm (Liu et al., 2017). The essence of channel pruning is to eliminate channels that contribute less to model performance so that network complexity can be reduced (Wang and He, 2021; Wu et al., 2020). In the channel pruning algorithm, the scaling factor γ is introduced into batch normalization (BN) layer. After training both weight parameters and scaling factor γ , all γ are counted and ranked to determine the maximum γ that satisfies the pruning rate requirement, which is used as the threshold for pruning. Channels smaller than the threshold are removed and do not participate in the connection of neurons. The channel pruning algorithm crops model parameters in a simple and effective way to improve model computation efficiency. Finally, the accuracy loss can be minimized by model fine-tuning to obtain a more compact model (Molchanov et al., 2019). The main steps are detailed as below:

(1) Sparse training.

Since the relationship between weights and inputs in deep learning networks is multiplicative and additive, if most of the network weights are zero or close to zero, removing this part has little impact on the final network output accuracy. The L1 regular constraint is applied to the scaling factor of BN layer of YOLOV5s grape clusters detection network

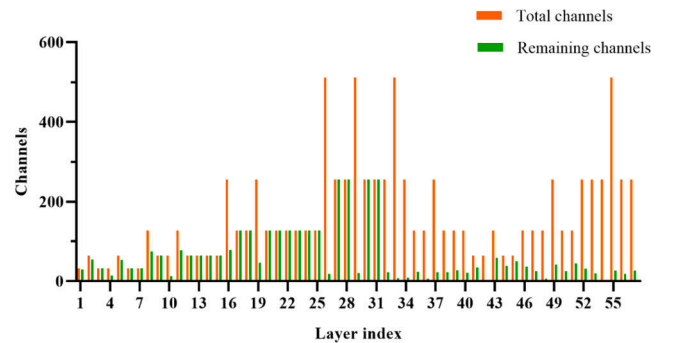


Fig. 6. Change of channels before and after model pruning.

to increase the constraint, thus the parameters are sparse, so that most of the learned network weights are close to zero. The loss function used in sparse training process is shown in Eq. (1).

$$L = \sum_{(x,y)} (f(x, W), y) + \lambda \sum_{\gamma \in \Gamma} g(\gamma) \quad (1)$$

where x and y are the training input and output parameters, respectively; W is the weight parameter vector to be learnt; $g(\gamma)$ is a penalty term on the scale factor γ ; λ is used as a penalty factor to balance the two terms.

For sparse training, λ is set to 0.002 and the remaining hyper-parameters are set to be the same as in normal training (see section 2.2.1). The distribution histogram of the coefficients of γ for the BN layers as epoch changes is shown in Fig. S1 in the supplementary material. As the sparse training epoch continues to increase, the center of the distribution of γ coefficients of all BN layers gradually approaches to 0, but not all of them decay to 0 and finally become gradually saturated, indicating that γ coefficients have completed the calibration for the channels with low contribution.

(2) Channel pruning.

After the sparse training is completed, a certain pruning rate is set and the channels with γ close to zero are pruned via pruning algorithm by removing all incoming and outgoing connections and the

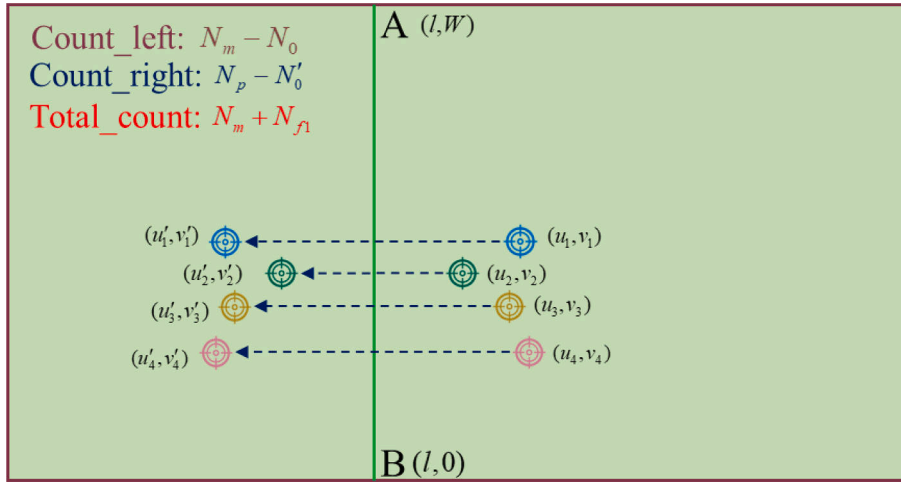


Fig. 7. Cross-line counting illustration. (AB is the counting line, (l, W) and $(l, 0)$ are the coordinates of points A and B respectively. The video size is $H \times W$, l is $2/5$ of H . (u_i, v_i) and (u'_i, v'_i) are coordinates of center point of the detection box in the i^{th} cluster that has not crossed and has crossed the count line, respectively. “Count_left” and “Count_right” represent the number of clusters crossing the count line in the leftward and rightward movement relative to the count line, respectively. “Total_count” represents the total number of clusters.

corresponding weights to generate a model that takes up less storage space and is more compact. The main flowchart of the pruning algorithm is shown in Fig. 5.

In this study, by considering the trade-off between model size and accuracy, the pruning rate was set to 0.85. The changes in the number of channels before and after model pruning are shown in Fig. 6. Detailed results are provided in Table S1 in the supplementary data. It can be seen that the number of channels in most convolutional layers is greatly reduced, and the number of channels in the whole model is reduced from the original 9504 to 3578, indicating that the number of channels is effectively reduced by the pruning algorithm, thus reducing model complexity.

(3) Fine-tuning the pruned model with improved NMS.

As a result of pruning, the higher pruning rate temporarily leads to a loss of model accuracy. However, the pruned network is able to overcome accuracy loss caused by pruning to some extent through subsequent fine-tuning process. The network was fine-tuned by loading the pruned YOLOv5s model weight parameters initially. In the fine-tuning stage, the learning rate, batch size and epoch were set to be 0.02, 16 and 100, respectively.

YOLOv5s solves the problem of multiple prediction boxes around a predicted object by using non-maximal suppression (NMS) during prediction. This method can effectively eliminate redundant bounding boxes by forcibly removing boxes with confidence levels less than a threshold and calculating the Intersection over Union (IoU) between the maximum confidence detection box and the remaining detection boxes in a region of the same class of object, while removing all detection boxes that are larger than a preset IoU threshold. However, in field conditions, there is an overlap between grape clusters due to grape cluster shape variations. Using this method will force to make the score of partially overlapping detection boxes zero, which will result in missed detection. Therefore, this study introduces Soft-NMS (Bodla et al., 2017) in YOLOv5s model prediction to improve the model's ability to detect overlapping clusters. When there are multiple predicted boxes around the cluster, the scores of the predicted boxes are multiplied by a weighting function that attenuates the scores of the neighboring boxes that overlap with the highest scoring box (Li et al., 2021). The more boxes overlap with the highest scoring box, the more severe the attenuation. The Gaussian function is chosen as the weighting function and is calculated as in Eq. (2).

$$s_i = s_i e^{-\frac{\text{iou}(M, b_i)^2}{\sigma}} \quad (2)$$

where s_i is score of the current prediction box, M is the current highest scoring prediction box, and b_i is a series of adjacent candidate boxes. When the candidate box i has a high overlap with the highest

scoring prediction box M , the Gaussian weighting function decays the candidate box i score instead of directly setting it to 0. Otherwise, the original score of the candidate box i will be maintained.

2.3. Online multiple object tracking of grape clusters by SORT algorithm

To achieve fast multiple object tracking of grape clusters under natural conditions, this study performs online multiple object tracking of clusters based on the SORT association algorithm (Bewley et al., 2016). The detailed steps are as follows.

(1) Each video frame is first obtained by video stream data processing algorithm, and then each cluster coordinate position is generated by the pruned YOLOv5s grape cluster detection model.

(2) State estimation of the object's position in the next video frame is done by Kalman filter (Bewley et al., 2016), which is mainly comprised of two phases including prediction and update. The prediction phase predicts position of the grape cluster in current frame based on position of the tracked cluster in previous frame; the update phase updates cluster position in prediction phase based on the detected position of the cluster in current frame (Wang et al., 2019).

In this study, an 8-dimensional vector $x = (u, v, r, h, \dot{u}, \dot{v}, \dot{r}, \dot{h})$ is used to model the state of grape cluster movement, (u, v, r, h) represent horizontal and vertical coordinates, aspect ratio and height of the center of the bounding box, and $(\dot{u}, \dot{v}, \dot{r}, \dot{h})$ is the derivative of (u, v, r, h) , which represents velocity information corresponding to the cluster in image coordinates. The state of the clusters was predicted by Eq. (3).

$$\begin{cases} \tilde{x}_t = F\tilde{x}_{t-1} \\ P_t = FP_{t-1}F^T + Q_{t-1} \end{cases} \quad (3)$$

where $\tilde{x}_t$ is estimated state of the cluster at frame t , F is the state transition matrix, $\tilde{x}_{t-1}$ is state of the cluster at frame $t-1$, P_t is the error covariance matrix estimation at frame t , P_{t-1} is error covariance matrix at frame $t-1$, and Q_{t-1} is the noise covariance matrix at frame $t-1$.

The movement state of the cluster is updated by Eq. (4) (Hamuda et al., 2018).

$$\begin{cases} K_{t+1} = P_t H^T (H P_t H^T + R)^{-1} \\ r_{t+1} = Z_{t+1} - H \tilde{x}_t \\ \tilde{x}_{t+1} = \tilde{x}_t + K_{t+1} r_{t+1} \\ P_{t+1} = (I - K_{t+1} H) P_t \end{cases} \quad (4)$$

where K_{t+1} is the Kalman filter gain, H is the observation matrix, R is the observation noise matrix, r_{t+1} is the residual between the observed and predicted positions of the cluster at frame $t+1$, Z_{t+1} is the cluster new observed location at frame $t+1$, $\tilde{x}_{t+1}$ is the estimated update of the

state of the cluster at frame $t + 1$, and I is the identity matrix.

(3) Cluster association matching. To perform multi-objective cluster association matching of video frame sequences, the IoU between the predicted box by Kalman filter and the actual detected box by the model is used as the cost matrix for consecutive frames, and the optimal solution of matching the predicted and detected results is sought by Hungarian algorithm (Kuhn, 1955) to achieve online tracking. It also ensures that each tracked cluster can only be matched with one detected cluster, and the Kalman filter is updated if the match is successful.

2.4. Counting grape clusters by cross-line counting

Due to irregular shape of grape clusters and complex background in the field (e.g., occlusion), the identity code (ID) assigned during tracking may switch, leading to over-counting. Therefore, this study counts the clusters by setting a counting line in the video, which is set at the position where the ratio of left to right area is about 2:3. Counting is performed by determining the relationship between the position of the count line and the coordinates of the detection box. If the intersection of the line connecting center points of the detection boxes of the same tracking object and the counting line exists between adjacent frames, the count is activated. Two counting modes are set according to different vehicle moving directions, and the counting modes can be automatically judged by the relative coordinate positions of detection boxes between adjacent frames. Fig. 7 shows the first counting mode as an example of the process of counting across lines. The two counting modes are described in detail as follows:

- (1) Cluster relative count line to the left moving pattern count step: (a) first initialize the number of detections on the left side of the counting line in 1st frame as N_0 . (b) from 2nd frame, count the number of clusters crossing the counting line from right to left in real time to accumulate on the basis of N_0 until $(T-1)^{th}$ frame, and the result is recorded as N_m . (c) count the number of detections on the right side of the counting line in the last frame as N_{fl} . The final count result is $N_m + N_{fl}$.
- (2) Similarly, the clusters are counted relative to the counting line in the rightward movement mode: (a) firstly, the number of detections on right side of the counting line is initialized in the first frame. (b) from the second frame, the number of clusters crossing the counting line from left to right is counted in real time and added on top of it until $(T-1)^{th}$ frame, and the result is recorded as N_p ; (c) the number of detections on the left side of the counting line is counted as N_{l2} in the last frame. The final count result is $N_p + N_{l2}$.

2.5. Experimental platform and evaluation metrics

All experiments in this study were conducted on a computer with a Linux (Ubuntu 20.04) operating system with a CPU of Intel(R) Core™ i9-11900 K (3.50 GHz), 32 GB of RAM, a GPU of NVIDIA RTX 3090 24 GB. The development environment is Python 3.7 and Pytorch version is 1.7.1.

Performance of grape cluster detection model was evaluated on test set by using F1 score and mAP. Precision (P) and recall (R) used to calculate the F1 score were determined by Eq. (5). The F1 score and mAP were calculated as in Eq. (6) and (7).

$$\begin{cases} P = \frac{TP}{TP + FP} \\ R = \frac{TP}{TP + FN} \end{cases} \quad (5)$$

$$F1 = \frac{2PR}{P + R} \quad (6)$$

Table 1

Performance comparison of YOLOv5s grape clusters detection model before and after pruning. (* indicates that Soft-NMS is used in the fine tuning).

Metrics	Original YOLOv5s	After pruning	After fine tuning	After fine tuning*
Parameters	7.0×10^6	1.5×10^6	1.5×10^6	1.5×10^6
FLOPs (G)	15.8	6.7	6.7	6.7
Model size (MB)	14.4	3.3	3.4	3.4
F1-score (%)	79.5	78.6	79.1	79.5
mAP (%)	82.5	81.7	82.1	82.3
Inference time (s)	6.8×10^{-3}	6.1×10^{-3}	6.1×10^{-3}	6.1×10^{-3}

$$mAP = \frac{\sum_{c=1}^C AP(c)}{C} \quad (7)$$

where TP , FP , and FN means true positive, false positive, and false negative, respectively. C represents the category, and in this study, $C = 1$. In AP calculation, TP is considered when the IoU between the detection box and the ground truth is greater than 0.5.

In addition, P_c (counting accuracy) was used to evaluate the performance of fruit counting in the video, calculated as in Eq. (8)

$$P_c = \left(1 - \frac{|N_g - N_a|}{N_g} \right) \times 100\% \quad (8)$$

where N_g and N_a are the ground truth and the proposed algorithmic count values of the video, respectively. The ground truth is obtained by two dedicated counters manually counting from the video.

3. Results

3.1. Model performance before and after pruning

In order to verify the impact of the channel pruning algorithm in this study on performance of grape cluster detection model, performance of YOLOv5s baseline model and the pruned model were compared on the image test set. The results are shown in Table 1. In particular, model efficiency is compared in terms of the number of model parameters, floating-point operations (FLOPs) and model size. FLOPs are used to measure model computational complexity. Model size includes not only number of parameters, but also network architecture information and optimizer information. From Table 1, it can be seen that after pruning, the amount of model parameters is reduced by 79 %, FLOPs are reduced by 58 %, model size is reduced by 11.1 MB (76 %), and the size of the pruned model is only 3.3 MB, which reduces inference time by 7×10^{-4} s compared to the original model. Although F1-score and mAP of the model decreased after pruning, the fine-tuning method was able to make model accuracy recover faster, and mAP of the fine-tuned model decreased by only 0.2 %. It is worth noting that model size is slightly increased after fine-tuning, but model accuracy is close to the original YOLOv5 model, which greatly reduces the model size and model complexity while ensuring model accuracy.

Table 2

Performance comparison of different object detection algorithms for grape clusters detection. (The best one is highlighted in bold.).

Algorithm	P (%)	R (%)	F1-score (%)	mAP (%)	Model size (MB)	Detection speed (f/s)
YOLOv4	87.84	64.22	74.00	80.11	256.3	39.8
YOLOv4-tiny	83.68	59.43	69.00	72.87	23.6	236.3
SSD 300	87.28	48.76	63.00	63.78	95.0	125.1
EfficientDet-D1	86.28	69.01	77.00	81.19	26.9	25.6
Ours	84.3	75.3	79.50	82.3	3.4	163.9

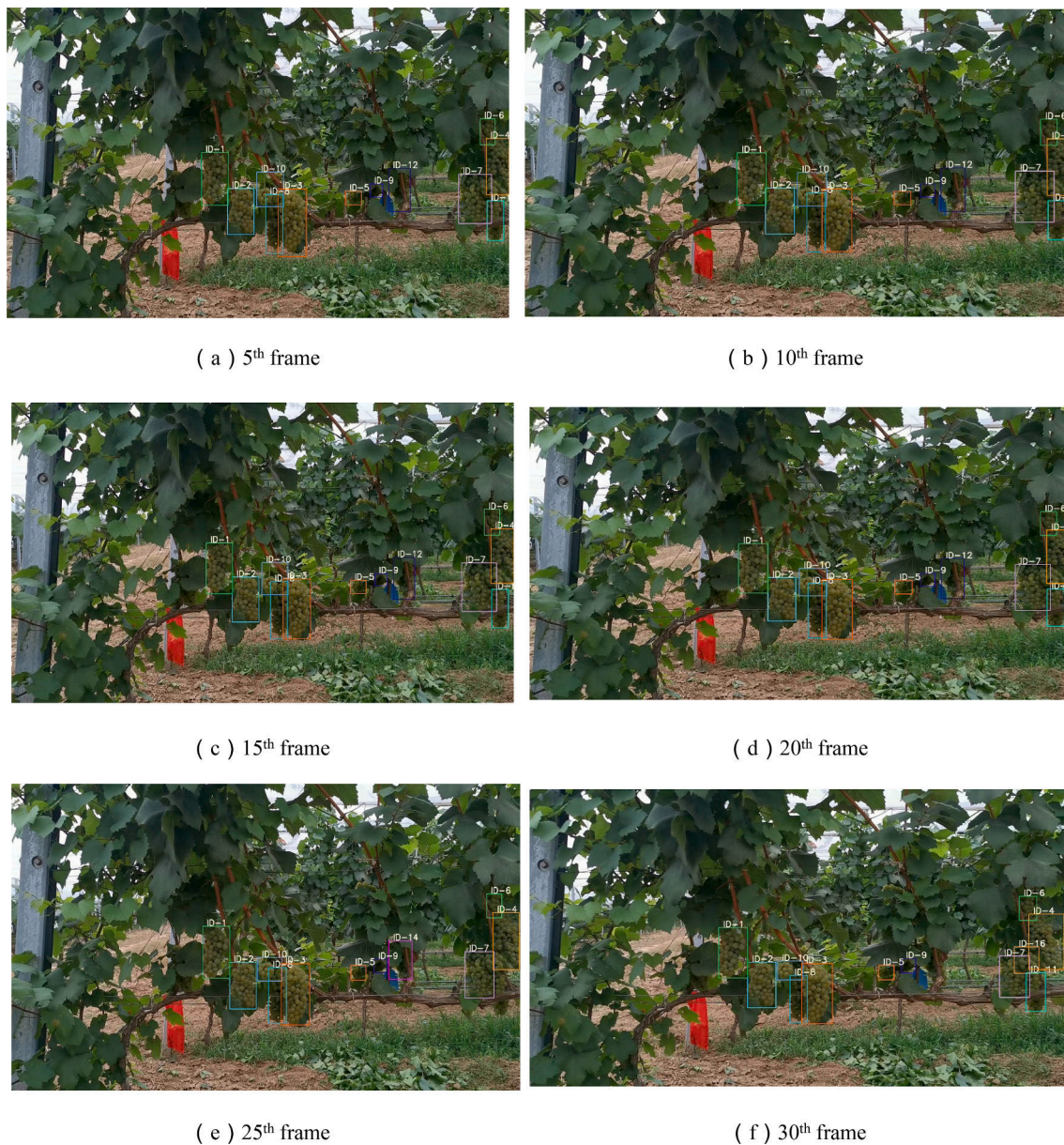


Fig. 8. Example of video V2 grape cluster multiple object tracking results. (The rectangular box represents the detected cluster and the number above represents the identity of the tracked cluster.).

3.2. Performance comparison of different object detection algorithms

To further validate the effectiveness of the proposed grape cluster detection method, four current widely used object detection algorithms, namely YOLOv4 (Bochkovskiy et al., 2020), YOLOv4-tiny, SSD (Liu et al., 2016), and EfficientDet-D1 (Tan et al., 2020), were trained by using the same training set. The performance of these five algorithms was evaluated under the same test set, and the obtained results were shown in Table 2. As can be seen from the table, recall, F1-score, and mAP of the proposed method are always higher than the other four algorithms, and the model size is the smallest. The recall of the proposed method is higher than YOLOv4, YOLOv4-tiny, SSD 300, and EfficientDet-D1 by 11.08 %, 15.87 %, 26.54 %, and 6.29 %, respectively. The F1-score is higher by 5.5 %, 10.5 %, 16.5 %, and 2.5 %, respectively. The mAP is higher by 2.19 %, 9.43 %, 18.52 %, and 1.11 %, respectively. The detection speed is 124.1f/s, 38.8f/s, and 138.3f/s higher than YOLOv4, SSD 300, and EfficientDet-D1, respectively. Although the detection speed of YOLOv4-tiny is faster than the proposed method, its

detection accuracy is lower and model size is larger. In addition, despite the fact that YOLOv4 has the highest precision among the five algorithms, its model size is also the largest among these five algorithms, and its detection speed is slower. It is noted that detection accuracy of EfficientDet-D1 is second only to the proposed method, but its model detection speed is the slowest. As a whole, the proposed method has better performance than the other four algorithms and can achieve accurate and fast identification of grape clusters in field conditions, which lays the foundation for the follow-up tracking and counting.

3.3. Multiple object tracking of grape clusters

Using the pruned YOLOv5s as a detector, the clusters in the video are identified and the corresponding bounding boxes are output. Combined with the SORT algorithm for multiple object tracking of grape clusters, the V2 clusters are moved to the left relative to the camera and some tracking results are shown in Fig. 7. During tracking, different instances of the detected clusters are assigned an ID as its identity. In subsequent

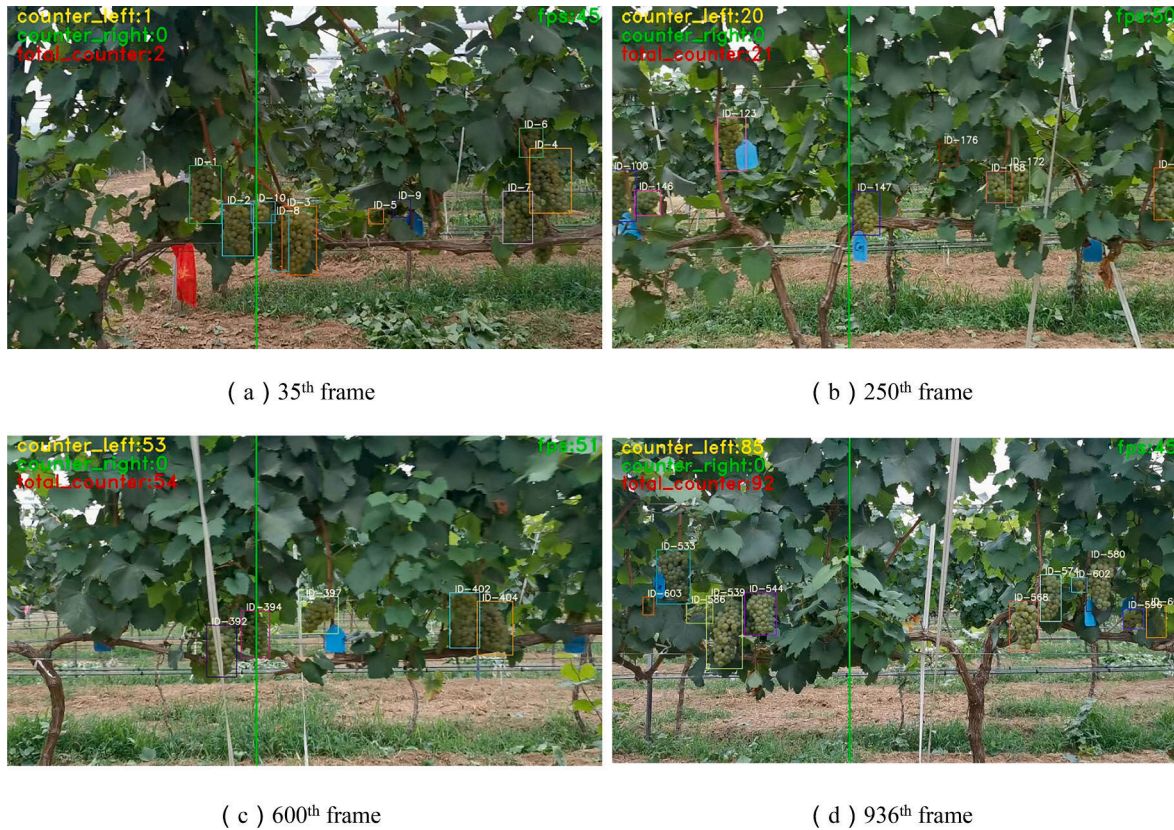


Fig. 9. V2 cross-line counting example results (Taking the first counting mode as an example, the green line at 2:3 in the figure is the count line, and the fps in the upper right corner represents the processing speed of the current frame. More details can be found in Fig. 6.).

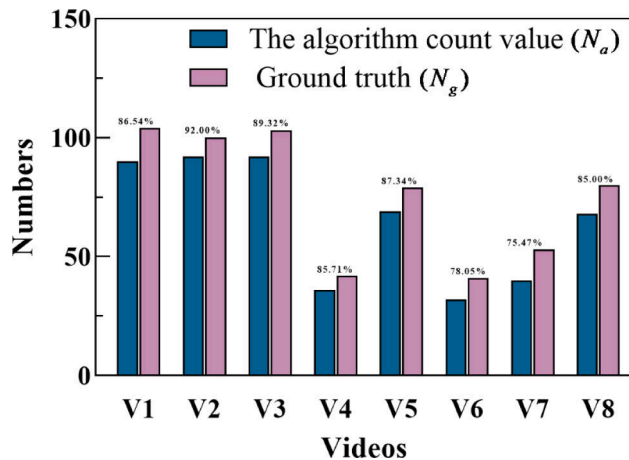


Fig. 10. Comparison of algorithmic counts and ground truth. (Percentage above the bar represents the P_c).

frames, if the ID does not switch, the tracking proves to be valid until it disappears from the field of view. As can be seen in Fig. 8, the ID switches between frames due to the clusters aggregating with each other, leading to intermittent detection problems for partially severely occluded same clusters, but the overall tracking results are promising (the ID of cluster #12 was switched at 25th frame due to severe occlusion. cluster #11 was able to be tracked accurately at 5th, 10th, 15th, and 20th frame, was not detected at 25th frame, but was retraced at 30th frame). Similarly, V1 clusters move to the right relative to the camera and the tracking results are shown in Fig. S2 of the supplementary material.

3.4. Grape cluster cross-line counting results

The counting performance of the proposed method was tested on 8 videos with different moving directions and different lengths. An example of real-time trace counting for V2 is available at <https://github.com/Nwaf-sl/Grape-track-count/blob/main/V2-result.mp4>. Correspondingly, Fig. 9 shows the results of real-time cross-line cluster counting in the first counting mode for the V2. In addition, partial results in the second counting mode are shown in Fig. S3 of the supplementary material. Although some cluster IDs switch during the tracking counting process leading to overestimation, the proposed cross-line counting method is promising for mitigating ID switching during the counting process.

Counts of the proposed algorithm test are compared with ground truth counts and the results are shown in Fig. 10. The counting accuracy (P_c) ranges from 75 % to 92 %, with an overall average P_c of 84.9 %. In particular, the average algorithm speed ranged from 43.1 to 50.4 FPS, meeting real-time online requirements. It is worth noting that while V6 and V7 accuracy is slightly lower than other videos, with an average P_c of 76.7 %, the remaining videos have the P_c above 85 %, achieving satisfactory results under field conditions.

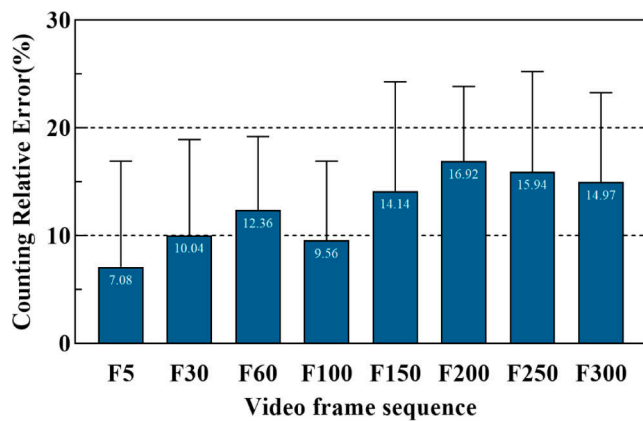
Also, the first 300 frames of each video were selected respectively, and 8 sequence segments of these 300 frames were analyzed. The algorithmic count results (N_a) were compared with the actual counting results on the ground obtained by the counters (N_g), and the results are shown in Table 3 and Fig. 11. For the first 5 frames, the algorithm counting results and the manual counting results basically remained the same, with an average relative error of 7.06 %. It can be noted that the first 5 frames had counts with 0, which was caused by our counting mode setting. Some of the videos did not initialize the detection of the target on one side of the counting line at 1th frame and had not reached the counting line during the movement of the cluster within the first few

Table 3

Comparison between algorithmic counting and real counting of different video clips.

Videos	Number of frames		F5	F30	F60	F100	F150	F200	F250	F300
V1	1027	N_a	6	6	8	13	18	21	25	32
		N_g	6	6	9	15	19	24	28	34
V2	936	N_a	0	1	4	10	15	18	21	26
		N_g	0	1	5	10	19	24	29	33
V3	1469	N_a	3	4	6	10	15	17	20	22
		N_g	3	5	7	12	18	21	23	26
V4	326	N_a	7	9	12	14	17	22	30	35
		N_g	6	8	11	13	17	24	30	36
V5	726	N_a	4	4	5	7	9	16	20	26
		N_g	5	5	6	8	12	20	25	33
V6	936	N_a	0	5	6	11	15	16	18	20
		N_g	0	5	6	11	17	19	22	24
V7	731	N_a	6	7	10	12	13	14	16	19
		N_g	5	6	10	15	18	19	22	26
V8	902	N_a	6	8	12	15	17	22	25	28
		N_g	6	9	13	16	18	24	28	31

(Fn indicates the count estimation within n frames where n = 5,30,60,100,150,200,250,300.).

**Fig. 11.** Average count relative error results for different test frame sequences of 8 videos.

frames. As the number of frames increases, the counting error fluctuates. This behavior may be due to the similarity of clusters and leaf colors, resulting in a trade-off between intermittent detections caused by false negatives and the overcounting problem associated with false positive detections, to a certain extent reducing the counting error of subsequent frames (e.g. V1 between frames 60 to 150 and V3 between frames 150 to 250). However, the maximum error does not exceed 17 %. It is worth mentioning that the algorithmic calculations and real counts remain almost identical for V1-V8 until 60 frames, with a maximum absolute error of no more than 1. Within the first 300 frames, V4 has the highest P_c , reaching 97.22 %. In addition, it is interesting to note that the algorithmic counts and the actual counts remain consistent for the first 100 frames for V6. However, from 100 to 150 frames both counting results show abrupt changes and are continuously accumulated in subsequent frames. This error due to detection false positives is maintained until 250 frames. Although a counting accuracy of 83.33 % was obtained for the first 300 frames, only an accuracy of 78.04 % was finally obtained for the entire video. In particular, while the V7 error maintained a brief balance between 150 and 200 frames, it continued to be increased later on, reaching a maximum in 300th frame, with a counting accuracy of 73.07 % in the first 300 frames, but ultimately obtaining an accuracy of 75.47 % on the entire video. These results may be related to the quality of the captured video and the inconsistent video length, and the complex and variable field conditions make some uncertainty in the counting results.

4. Discussion

4.1. Effect of different field conditions on grape clusters detection

The complex and changing natural conditions in the field may have an adverse impact on accurate detection of grape clusters. The detection performance of the pruned YOLOv5s model was compared on test set under four different conditions, and the results are shown in Fig. 12. Although the pruned model is smaller in size and less complex in network architecture, it does not significantly reduce the ability to accurately identify grape clusters under complex background conditions. The grape clusters were accurately detected under both natural light (Fig. 12a), smooth light (Fig. 12b), backlight (Fig. 12c) and different acquisition distance conditions (Fig. 12d). This may be due to data enhancement of the training set, where the light and dark changes in brightness and contrast may truly reflect the changes in field lighting. Although the proposed model was able to achieve 82.3 % in mAP for cluster recognition on test set, there is still some uncertainty in the results obtained, as the dataset is manually annotated. Due to extreme occlusions, overlaps between clusters and the fact that some clusters are at image edge, the model may detect clusters that are not fully annotated, causing the presence of false positives. In a previous study by Li et al. (2021) on table grapes recognition, an attention mechanism was introduced in YOLOv4-tiny to improve model recognition accuracy under complex growth conditions. However, potentially, it also introduces additional time overhead to the model. Moreover, there are some differences in cultivation methods of table grapes and wine grapes, and the location and status of grape clusters in the grapevine distribution are also important factors affecting detection results. For the dataset in this study, owing to the variations in camera capture distance, there are significantly fewer examples of clusters in the images collected in 2020 than 2021. However, for the same-size images, the individual cluster in the images collected in 2020 occupy a larger pixel area and can be more easily detected by the model. This model trained at different collection distances is highly robust and does not vary significantly in the field due to changes in distance from grape rows by tractor platform or robot with camera.

4.2. Effect of tracking errors on counting results

The SORT-based multiple object tracking algorithm is subject to errors in cluster association due to large inter-cluster shape variations, extreme occlusions, and camera pose variations. Fig. 13 shows several examples of tracking errors. The main tracking error scenarios are (1) ID switching due to extreme trunk and leaf occlusion (Fig. 13a). (2) Intermittent tracking and mis-matching caused by detection errors

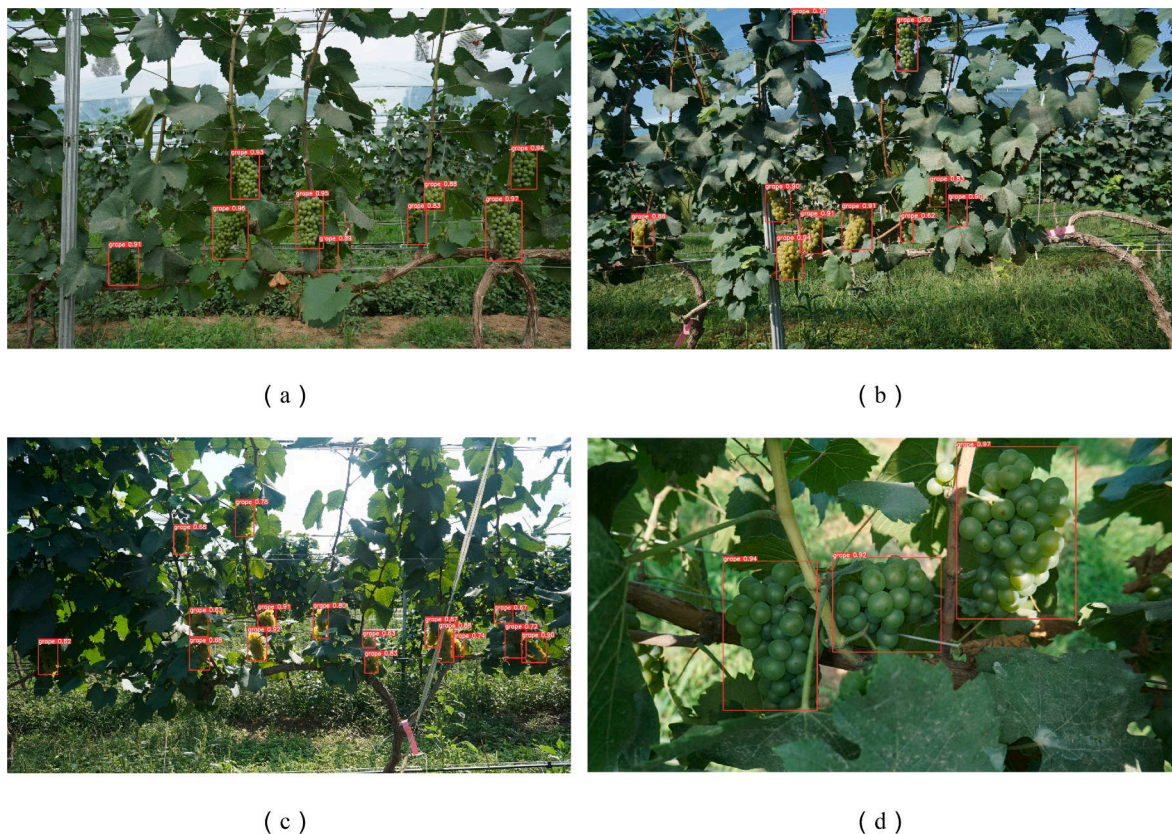


Fig. 12. Detection results of the pruned YOLOV5s model under different natural conditions. (a), (b) and (c) are results under natural light, smooth light and backlight conditions, respectively. (d) indicates results under different acquisition distances.

(Fig. 13b). (3) ID switching caused by cluster's overlapping and intermediate frame missed detection (Fig. 13c). During the tracking process, the number of IDs assigned to the tracking is higher than the actual count value. The cluster matching error between frames, where the same object is assigned a new ID multiple times, causes the ID to change (Fig. 13a). In a previous similar study by Vasconez et al (2020), which counted three fruits, Hass avocado, lemon, and apples, by the number of IDs tracked, also reported higher algorithmic count estimates. Compared to the three relatively regular shaped fruits, tracking is very challenging with a large variability in shape among grape clusters and cluster overlapping occlusion. It is highly likely that trajectory error matches or even mismatches occur in cluster data association, and the number of grape cluster instances tracked is often higher than the actual number. Surprisingly, this study was able to greatly alleviate this situation by setting up counting line. The ID switching does not affect the result of the count until the tracked target cluster reaches the count line and after it crosses the count line. This is due to the fact that we set the count to be valid only when center point of the tracked target crosses count line. However, there are special cases where persistent miss detection generated before and after the cluster crosses the count line in the vicinity can cause the cluster to cross the count line directly and not be detected. Although counting errors exist, the counting line greatly reduces the observation range of the tracking error, improves the probability of the cluster being counted effectively, and better alleviates the wrong counting result caused by frequent ID switching.

Further, linear regression analysis results of real counts and algorithmic counts are shown in Fig. 14. R^2 of 0.9905 indicates that the proposed method has a significant linear correlation real count. This implies that the number of clusters estimated by the algorithm is similar to the ground truth number, which is significant for yield prediction and yield mapping related applications.

4.3. Effect of different counting line positions on counting results

In order to investigate the effect of counting line position on counting results, we set two other counting line positions, 3:2 and 1:1 for the left and right areas of the counting window, and compared them with the previously 2:3 position, where the results are shown in Fig. 15. Surprisingly, the effect of counting line position is very slight, and the average standard deviation of the three different counting line positions is about 1.70 on the 8 test videos, which is almost negligible. This also indicates that the setting of count line position in this study does not make a significant difference to the cluster counts within a certain range. However, for very extreme cases, center point of the target detection frame falls exactly at the position of count line during the initialization of the first frame or during the counting on the count line side of the last frame, which may cause a counting error. In addition, due to the uncertainty of intermittent tracking, it can potentially occur at every position of the counting window. For the same tracking target, tracking loss may not have occurred at one point in time, but over time, it is undoubtedly valid to be counted during the time difference between the two when it is not tracked due to occlusion or a change in camera pose at another point in time. The position of count line setting essentially reflects the temporal order in which the tracked targets are spatially and sequentially counted. The difference in count line position causes the same target to cross the count line with different time consumption, which also means that the location of the valid count event detected by the count line occurs differently. It is worth mentioning that the counting results obtained by V2 video remain exactly the same for the three different count line positions, which may be related to a potential equilibrium relationship between the variation of count line positions and intermittent tracking.

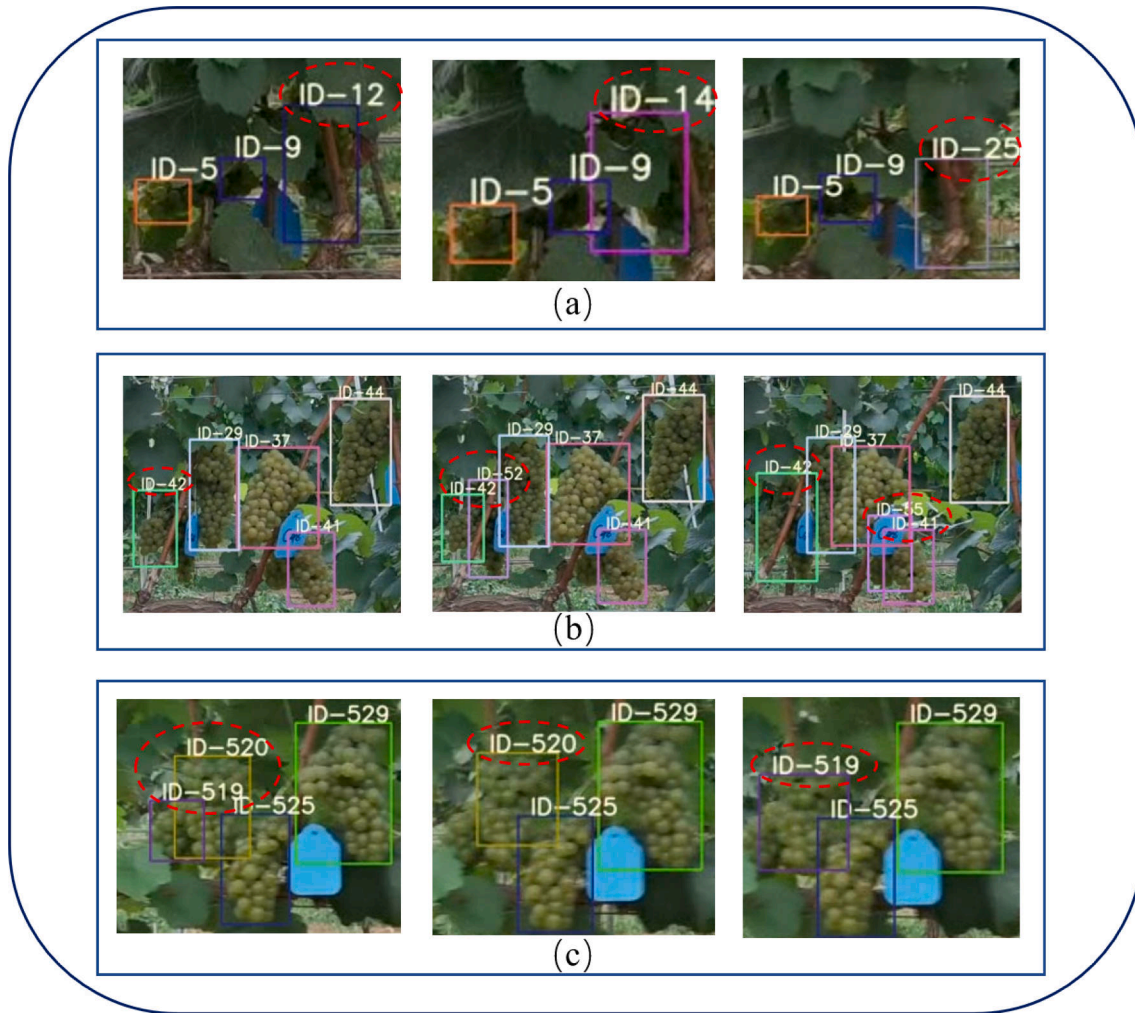


Fig. 13. Example of tracking error. (a) ID switching caused by extreme occlusion. (b) Faulty tracking due to detection error. (c) ID switching due to cluster overlap and intermediate frame miss detection. (The red dashed circle highlights the cluster where the tracking error occurred).

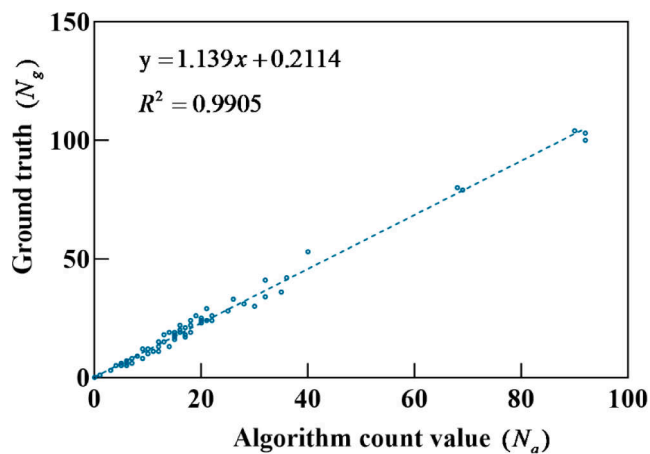


Fig. 14. Regression curve between algorithmic counts and ground truth counts of different video clips.

4.4. Limitations and future work

The proposed tracking and counting method based on channel pruning YOLOv5 in this study enables accurate and rapid counting of grape clusters in field conditions. However, there is room for further

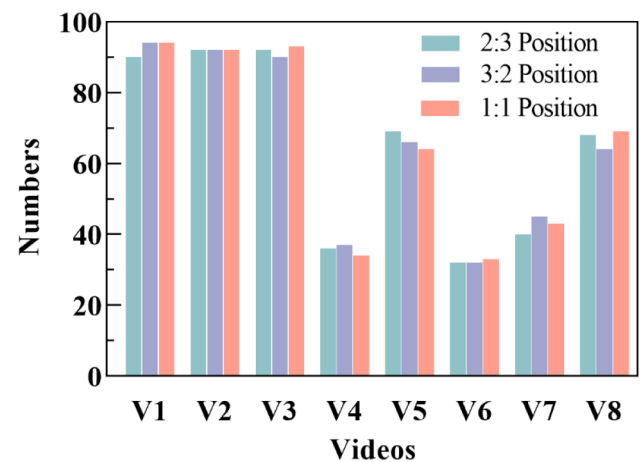


Fig. 15. Comparison of counting results for different counting line positions.

improvement, which is summarized as below.

In commercial vineyards, partial clusters appear on both sides of the vine, whereas the proposed method in this study only considers clusters visible from one side of grape row. These clusters may be double counted due to the change in camera view during vehicle movement on both

sides of the row. Therefore, future work will improve orchard yield estimation performance by developing a method that is able to identify clusters on both sides by using a depth camera, combined with the proposed tracking and counting algorithm to reject the same cluster with repeated counts on both sides.

In addition, due to the complex field conditions, the offset of video images caused by sudden changes in speed of moving platforms can lead to the loss of cluster tracking. Even though it can be effectively mitigated by setting count line, we plan to consider platform moving speed in data association matching of tracking in future studies to reduce the negative effects of video jitter.

From the results in Section 3.1, the model inference time does not change significantly before and after pruning, which is related to the high performance of the computing device in the study. However, the proposed method is also suitable for more ordinary hardware and is able to deliver good performance even for devices under 10G FLOPS (floating-point operations per second). Further, the pruned model is more lightweight, only 3.4 MB, and can process up to 50.4 FPS on average on the video in combination with cluster tracking counting method, which means that the proposed method can be deployed in small mobile terminals for orchard estimation robots in the future.

5. Conclusions

To achieve fast and accurate counting of grape clusters in field conditions for orchard yield estimation. In this study, channel pruning YOLOv5s was first conducted, where the number of model parameters, model size and FLOPs were reduced by 79 %, 76 % and 58 %, respectively. This approach effectively reduced model complexity with a model size of only 3.4 MB. In addition, Soft-NMS was also introduced in prediction stage to improve detection performance for overlapping grape clusters. The mAP and F1-scores on image test set reach 82.3 % and 79.5 %, respectively, and the average inference time per image is only 6.1 ms. Comparative results against four state-of-the-art object detection algorithms showed that the proposed model could not only improve model efficiency but also guaranteed model detection accuracy. Secondly, the object detection results were combined with SORT algorithm for online cluster tracking via video data, and the concept of counting line was introduced to set two counting modes according to vehicle movement direction to achieve real-time grape cluster counting. Finally, test results on eight videos showed that the average counting accuracy of the proposed algorithm could reach 84.9 %, and the correlation coefficient with manual counting reached 0.9905. The average speed of the proposed method can reach up to 50.4 FPS, which is able to meet the real-time requirements and provides timely technical support for online yield estimation and accurate crop management in orchards.

CRedit authorship contribution statement

Lei Shen: Conceptualization, Methodology, Software, Formal analysis, Resources, Visualization, Writing – original draft. **Jinya Su:** Formal analysis, Writing – review & editing. **Runtian He:** Formal analysis, Investigation. **Lijie Song:** Formal analysis, Investigation. **Rong Huang:** Resources, Investigation. **Yulin Fang:** Resources. **Yuyang Song:** Resources, Supervision. **Baofeng Su:** Project administration, Supervision, Funding acquisition, Writing – review & editing.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work was supported by the Key R&D Projects of Shaanxi Province, China (Grant No. 2021NY-041) and the Key R&D Program of Ningxia Hui Autonomous Region, China (Grant No. 2021BEF02017). The authors would like to thank the anonymous reviewers for their helpful comments and suggestions.

Appendix A. Supplementary data

Supplementary data to this article can be found online at <https://doi.org/10.1016/j.compag.2023.107662>.

References

- Aquino, A., Millan, B., Diago, M.-P., Tardaguila, J., 2018. Automated early yield prediction in vineyards from on-the-go image acquisition. *Comput. Electron. Agric.* 144, 26–36. <https://doi.org/10.1016/j.compag.2017.11.026>.
- Bargoti, S., Underwood, J., 2017. Deep fruit detection in orchards, 2017 IEEE international conference on robotics and automation (ICRA). IEEE, pp. 3626–3633. <http://doi.org/10.1109/ICRA.2017.7989417>.
- Barriguhina, A., de Castro Neto, M., Gil, A., 2021. Vineyard yield estimation, prediction, and forecasting: A systematic literature review. *Agronomy* 11 (9), 1789. <https://doi.org/10.3390/agronomy11091789>.
- Bewley, A., Ge, Z., Ott, L., Ramos, F., Upcroft, B., 2016. Simple online and realtime tracking, 2016 IEEE international conference on image processing (ICIP). IEEE, pp. 3464–3468. <http://doi.org/10.1109/ICIP.2016.7533003>.
- Bochkovskiy, A., Wang, C.-Y., Liao, H.-Y.M., 2020. YOLOv4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*.
- Bodla, N., Singh, B., Chellappa, R., Davis, L.S., 2017. Soft-NMS—improving object detection with one line of code, Proceedings of the IEEE international conference on computer vision, pp. 5561–5569. <http://doi.org/10.1109/ICCV.2017.593>.
- Dorj, U.-O., Lee, M., Yun, S.-S., 2017. An yield estimation in citrus orchards via fruit detection and counting using image processing. *Comput. Electron. Agric.* 140, 103–112. <https://doi.org/10.1016/j.compag.2017.05.019>.
- Dunn, G.M., Martin, S.R., 2004. Yield prediction from digital image analysis: A technique with potential for vineyard assessments prior to harvest. *Aust. J. Grape Wine Res.* 10 (3), 196–198. <https://doi.org/10.1111/J.1755-0238.2004.TB00022.X>.
- Fernández-González, M., Escuredo, O., Rodríguez-Rajo, F., Aira, M., Jato, V., 2011. Prediction of grape production by grapevine cultivar Godello in north-west Spain. *J. Agric. Sci.* 149 (6), 725–736. <https://doi.org/10.1017/S0021859611000244>.
- Fu, L., Feng, Y., Wu, J., Liu, Z., Gao, F., Majeed, Y., Al-Mallahi, A., Zhang, Q., Li, R., Cui, Y., 2021. Fast and accurate detection of kiwifruit in orchard using improved YOLOv3-tiny model. *Precis. Agric.* 22 (3), 754–776. <https://doi.org/10.1007/S11119-020-09754-Y>.
- Gao, F., Fang, W., Sun, X., Wu, Z., Zhao, G., Li, G., Li, R., Fu, L., Zhang, Q., 2022. A novel apple fruit detection and counting methodology based on deep learning and trunk tracking in modern orchard. *Comput. Electron. Agric.* 197, 107000 <https://doi.org/10.1016/j.compag.2022.107000>.
- Hamuda, E., Mc Ginley, B., Glavin, M., Jones, E., 2018. Improved image processing-based crop detection using Kalman filtering and the Hungarian algorithm. *Comput. Electron. Agric.* 148, 37–44. <https://doi.org/10.1016/j.compag.2018.02.027>.
- He, K., Gkioxari, G., Dollár, P., Girshick, R., 2020. Mask R-CNN. *IEEE Trans. Pattern Anal. Mach. Intell.* 42 (2), 386–397.
- Herrero-Huerta, M., González-Aguilera, D., Rodríguez-Gonzálvez, P., Hernández-López, D., 2015. Vineyard yield estimation by automatic 3D bunch modelling in field conditions. *Comput. Electron. Agric.* 110, 17–26. <https://doi.org/10.1016/J.COMPA.2014.10.003>.
- Jocher, G., Stoken, A., Borovec, J., Chaurasia, A., Changyu, L., 2020. ultralytics/yolov5. Github Repository, YOLOv5.
- Kamilaris, A., Prenafeta-Boldú, F.X., 2018. Deep learning in agriculture: A survey. *Comput. Electron. Agric.* 147, 70–90. <https://doi.org/10.1016/j.compag.2018.02.016>.
- Koirala, A., Walsh, K., Wang, Z., McCarthy, C., 2019. Deep learning for real-time fruit detection and orchard fruit load estimation: Benchmarking of ‘MangoYOLO’. *Precis. Agric.* 20 (6), 1107–1135. <https://doi.org/10.1007/s11119-019-09642-0>.
- Kuhn, H.W., 1955. The Hungarian method for the assignment problem. *Naval research logistics quarterly* 2 (1–2), 83–97. https://doi.org/10.1007/978-3-540-68279-0_2.
- Laurent, C., Oger, B., Taylor, J.A., Scholasch, T., Metay, A., Tisseyre, B., 2021. A review of the issues, methods and perspectives for yield estimation, prediction and forecasting in viticulture. *Eur. J. Agron.* 130, 126339 <https://doi.org/10.1016/J.EJA.2021.126339>.
- LeCun, Y., Bengio, Y., Hinton, G., 2015. Deep learning. *Deep learning. nature* 521 (7553), 436–444.
- Li, H., Li, C., Li, G., Chen, L., 2021. A real-time table grape detection method based on improved YOLOv4-tiny network in complex background. *Biosyst. Eng.* 212, 347–359. <https://doi.org/10.1016/j.biosystemseng.2021.11.011>.
- Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, C.L., 2014. Microsoft coco: Common objects in context. European conference on computer vision. Springer 740–755. https://doi.org/10.1007/978-3-319-10602-1_48.

- Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., Berg, A.C., 2016. Ssd: Single shot multibox detector. *European conference on computer vision*. Springer 21–37. https://doi.org/10.1007/978-3-319-46448-0_2.
- Liu, X., Chen, S.W., Liu, C., Shivakumar, S.S., Das, J., Taylor, C.J., Underwood, J., Kumar, V., 2019. Monocular camera based fruit counting and mapping with semantic data association. *IEEE Rob. Autom. Lett.* 4 (3), 2296–2303. <https://doi.org/10.1109/LRA.2019.2901987>.
- Liu, Z., Li, J., Shen, Z., Huang, G., Yan, S., Zhang, C., 2017. Learning efficient convolutional networks through network slimming. *Proceedings of the IEEE international conference on computer vision*, pp. 2736–2744. <http://doi.org/10.1109/ICCV.2017.298>.
- Liu, X., Chen, S.W., Aditya, S., Sivakumar, N., Dcunha, S., Qu, C., Taylor, C.J., Das, J., Kumar, V., 2018. Robust fruit counting: Combining deep learning, tracking, and structure from motion, 2018 IEEE/RSJ international conference on intelligent robots and systems (IROS). IEEE, pp. 1045–1052. <http://doi.org/10.1109/IROS.2018.8594239>.
- Liu, G., Nouaze, J.C., Touko Mbouembe, P.L., Kim, J.H., 2020. YOLO-tomato: A robust algorithm for tomato detection based on YOLOv3. *Sensors* 20 (7), 2145. <https://doi.org/10.3390/s20072145>.
- Molchanov, P., Mallya, A., Tyree, S., Frosio, I., Kautz, J., 2019. Importance estimation for neural network pruning. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 11264–11272. <http://doi.org/10.1109/CVPR.2019.01152>.
- Parvathi, S., Selvi, S.T., 2021. Detection of maturity stages of coconuts in complex background using Faster R-CNN model. *Biosyst. Eng.* 202, 119–132. <https://doi.org/10.1016/j.biosystemseng.2020.12.002>.
- Redmon, J., Divvala, S., Girshick, R., Farhadi, A., 2016. You only look once: Unified, real-time object detection. *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 779–788. <http://doi.org/10.1109/CVPR.2016.91>.
- Ren, S., He, K., Girshick, R., Sun, J., 2017. Faster r-cnn: Towards real-time object detection with region proposal networks. *Adv. Neural Inf. Proces. Syst.* 39 (6), 1137–1149.
- Russell, B.C., Torralba, A., Murphy, K.P., Freeman, W.T., 2008. LabelMe: a database and web-based tool for image annotation. *Int. J. Comput. Vis.* 77 (1), 157–173. <https://doi.org/10.1007/s11263-007-0090-8>.
- Santos, T.T., de Souza, L.L., dos Santos, A.A., Avila, S., 2020. Grape detection, segmentation, and tracking using deep neural networks and three-dimensional association. *Comput. Electron. Agric.* 170, 105247. <https://doi.org/10.1016/J.COMPAG.2020.105247>.
- Shen, L., Su, J., Huang, R., Quan, W., Song, Y., Fang, Y., Su, B., 2022. Fusing attention mechanism with Mask R-CNN for instance segmentation of grape cluster in the field. *Front. Plant Sci.* <https://doi.org/10.3389/fpls.2022.934450>.
- Stein, M., Bargoti, S., Underwood, J., 2016. Image based mango fruit detection, localisation and yield estimation using multiple view geometry. *Sensors* 16 (11), 1915. <https://doi.org/10.3390/s16111915>.
- Tan, M., Pang, R., Le, Q.V., 2020. Efficientdet: Scalable and efficient object detection. *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 10781–10790. <http://doi.org/10.1109/cvpr42600.2020.01079>.
- Tian, Y., Yang, G., Wang, Z., Wang, H., Li, E., Liang, Z., 2019. Apple detection during different growth stages in orchards using the improved YOLO-V3 model. *Comput. Electron. Agric.* 157, 417–426. <https://doi.org/10.1016/j.compag.2019.01.012>.
- Vasconez, J.P., Delpiano, J., Vougioukas, S., Cheein, F.A., 2020. Comparison of convolutional neural networks in fruit detection and counting: A comprehensive evaluation. *Comput. Electron. Agric.* 173, 105348. <https://doi.org/10.1016/j.compag.2020.105348>.
- Wang, C.-Y., Liao, H.-Y.M., Wu, Y.-H., Chen, P.-Y., Hsieh, J.-W., Yeh, I.-H., 2020. CSPNet: A new backbone that can enhance learning capability of CNN. *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops*, pp. 390–391. <http://doi.org/10.1109/CVPRW50498.2020.00203>.
- Wang, D., He, D., 2021. Channel pruned YOLO V5s-based deep learning approach for rapid and accurate apple fruitlet detection before fruit thinning. *Biosyst. Eng.* 210, 271–281. <https://doi.org/10.1016/j.biosystemseng.2021.08.015>.
- Wang, D., He, D., 2022. Fusion of Mask RCNN and attention mechanism for instance segmentation of apples under complex background. *Comput. Electron. Agric.* 196, 106864. <https://doi.org/10.1016/j.compag.2022.106864>.
- Wang, Z., Walsh, K., Koirala, A., 2019. Mango fruit load estimation using a video based MangoYOLO—Kalman filter—hungarian algorithm method. *Sensors* 19 (12), 2742. <https://doi.org/10.3390/s19122742>.
- Wu, D., Lv, S., Jiang, M., Song, H., 2020. Using channel pruning-based YOLO v4 deep learning algorithm for the real-time and accurate detection of apple flowers in natural environments. *Comput. Electron. Agric.* 178, 105742. <https://doi.org/10.1016/J.COMPAG.2020.105742>.
- Yi, D., Su, J., Chen, W.-H., 2021. Probabilistic faster R-CNN with stochastic region proposing: Towards object detection and recognition in remote sensing imagery. *Neurocomputing* 459, 290–301. <https://doi.org/10.1016/j.neucom.2021.06.072>.
- Zabawa, L., Kicherer, A., Klingbeil, L., Töpfer, R., Kuhlmann, H., Roscher, R., 2020. Counting of grapevine berries in images via semantic segmentation using convolutional neural networks. *ISPRS J. Photogramm. Remote Sens.* 164, 73–83. <https://doi.org/10.1016/j.isprsjprs.2020.04.002>.
- Zheng, Z., Wang, P., Liu, W., Li, J., Ye, R., Ren, D., 2020. Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression. *AAAI* 34 (07), 12993–13000.